

Technical & Product Overview

A plain-language walkthrough of what's been built in Phase 1, how the product is architected from here through Phase 3, and what to expect when you click through it yourself.

[Get started](#)

✦ Built for Nigerian schools & families

The school-to-home connection your family can keep up with.

See what your child is learning, what's due, and what school said, all without digging through group chats.

[Get started](#)[Why Kithnest](#)



Why Kithnest

Every existing player builds for the teacher or the school office first. Kithnest inverts that: the parent is the primary user, on three non-negotiables.

Prepared for

Product Team

Prepared by

Team 6, Intellect Fellowship

Date

1 August 2026

Stage

Phase 1 live · Phase 2–3 planned

START HERE

What this document is

A companion to the live build, in plain language

This document walks through what's been built in Phase 1 of Kithnest, why it's built the way it is, and what to expect when you click through it yourself. It also lays out how the product is planned to be architected across Phase 2 and Phase 3, the backend, data, messaging, hosting, and AI, so the product team has a full picture, not just the part that's clickable today. We've kept it in plain language throughout; where a real technology choice matters, we name it and explain what it does, without assuming an engineering background.

What Phase 1 is, and isn't

Phase 1 is a fully working, fully clickable version of Kithnest. Every button, tab, and screen responds the way it would in a real product. What's different from a finished product is what's behind the screen: there's no live database, no real school signing in, and no real text messages being sent yet.

Instead, the app is populated with realistic sample data, a fictional school, a fictional parent, and a fictional class, so it looks and behaves exactly like the real thing would.

Think of it as a fully built stage set: everything you can see and touch is real, but nothing is wired up to the outside world yet. That's deliberate, it lets us test whether the experience feels right before we invest in the backend systems that make it truly live.



WHY KITHNEST EXISTS

The idea behind Kithnest

Every existing school app on the market, ClassDojo, Teachmint, Fedena, Toddle, was designed for the teacher or the school office first. Parents get whatever's left over, usually a one-way notification feed nobody checks.

Kithnest starts from the opposite direction: **the parent is the primary user**, and the school is the customer paying to give every parent in their school a better experience.

Three commitments show up on nearly every screen we've built:



See it, don't decode it

What a child is learning and what's due should be obvious at a glance, not buried in a teacher's shorthand or a long text message.



Built for real networks

Nigerian connectivity is inconsistent. Nothing important should require a live connection to be seen, only to update.



SMS isn't a fallback

A meaningful share of parents will never open an app. Kithnest is designed to reach them anyway, by text.

Everything on the following pages, the visuals, the interactions, even small details like an offline banner, exists specifically to serve one of these three commitments. None of it is decoration.

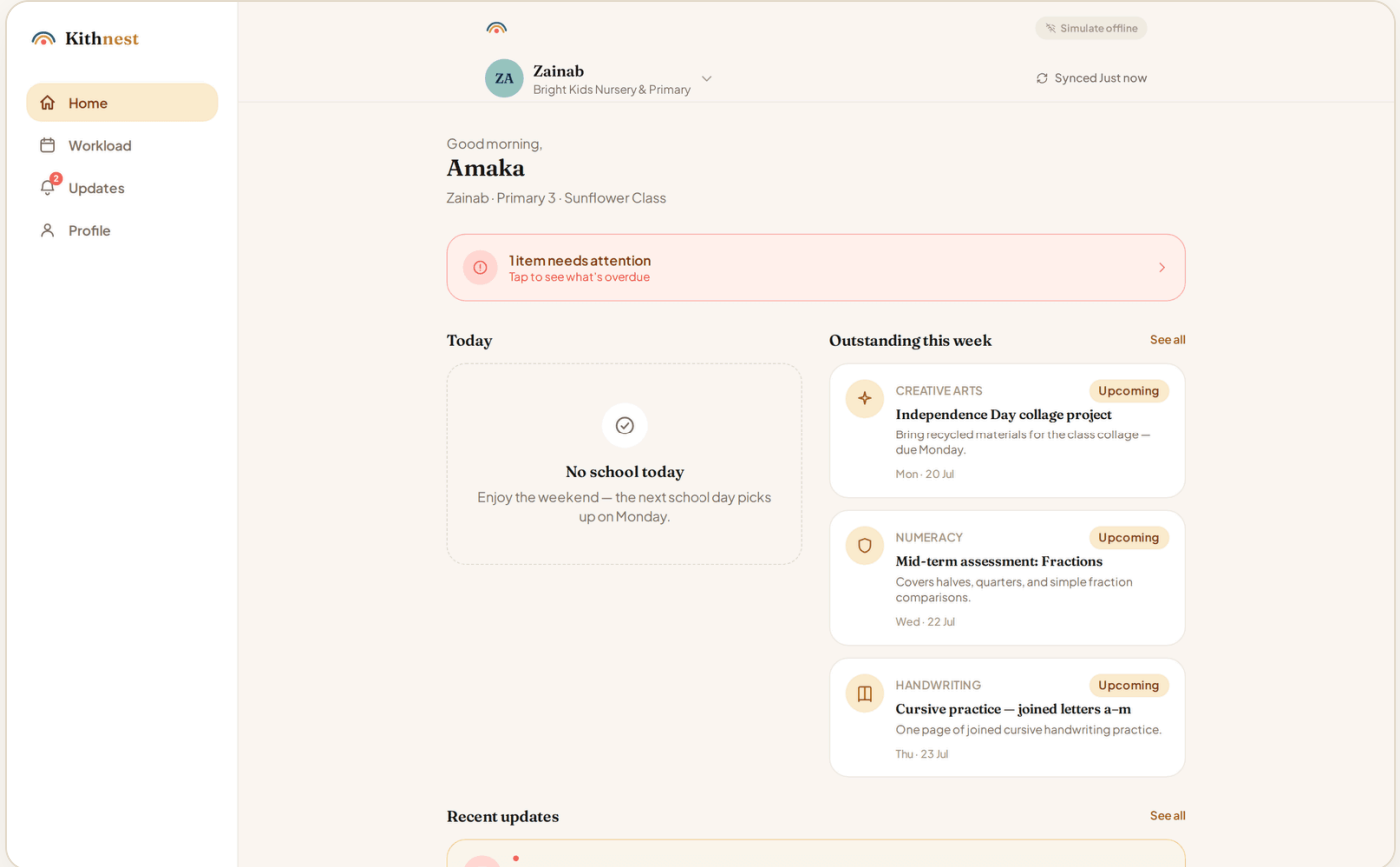


WALKTHROUGH · PART 1

Getting in, and the daily check-in

The two screens a parent sees most often

Signing in. A parent enters their school's code and their own phone number, nothing more. No passwords to forget, no long forms, just enough friction to feel secure and no more.

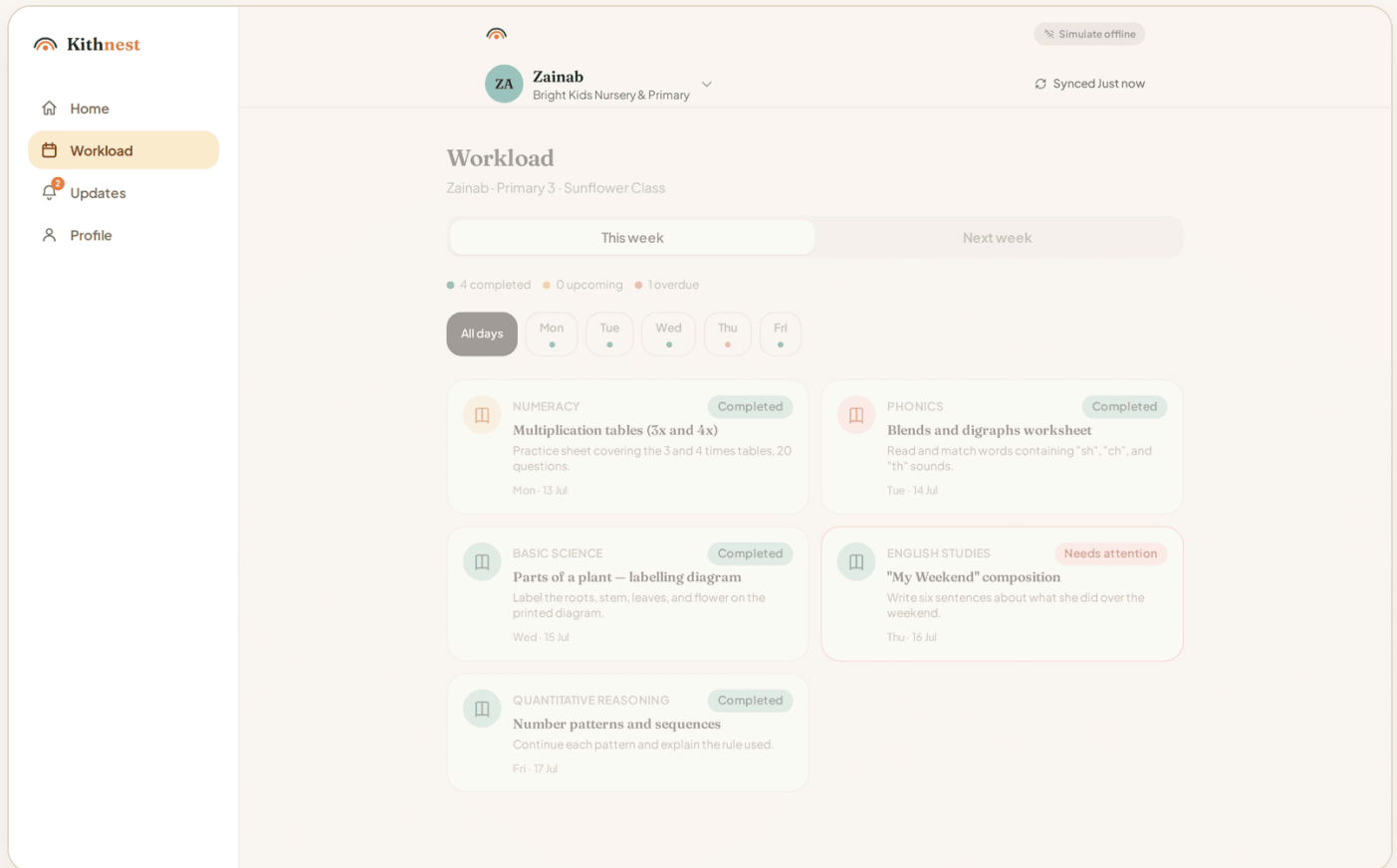


The daily check-in. The moment a parent logs in, they see a snapshot of their child's day: what's due today, what's overdue, and what's coming up this week, plus the most recent updates from school. This is designed to feel like opening a banking app: quick, reassuring, and immediately useful.

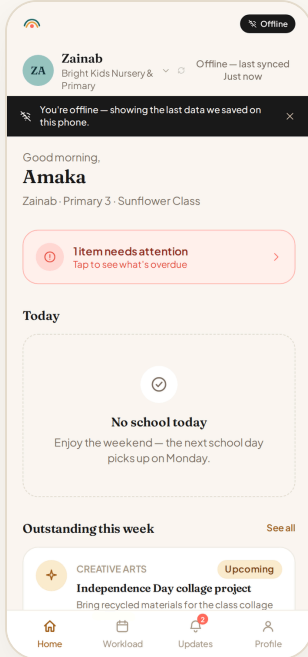
WALKTHROUGH · PART 2

Seeing the week, staying informed offline

Two of the three product commitments, made visible



Seeing the week, not a list. Instead of a wall of text, a parent sees their child's week laid out visually, by day, with colour-coded status (completed, upcoming, needs attention). It answers "is my child on top of things?" in one glance, no decoding required.



Staying informed, even offline. Nigerian connectivity drops in and out constantly. When it does, Kithnest shows a clear "you're offline" banner, and everything already loaded stays fully visible and usable, nothing disappears or breaks. It's a visual preview of the offline-first behaviour we'll build for real in Phase 2.

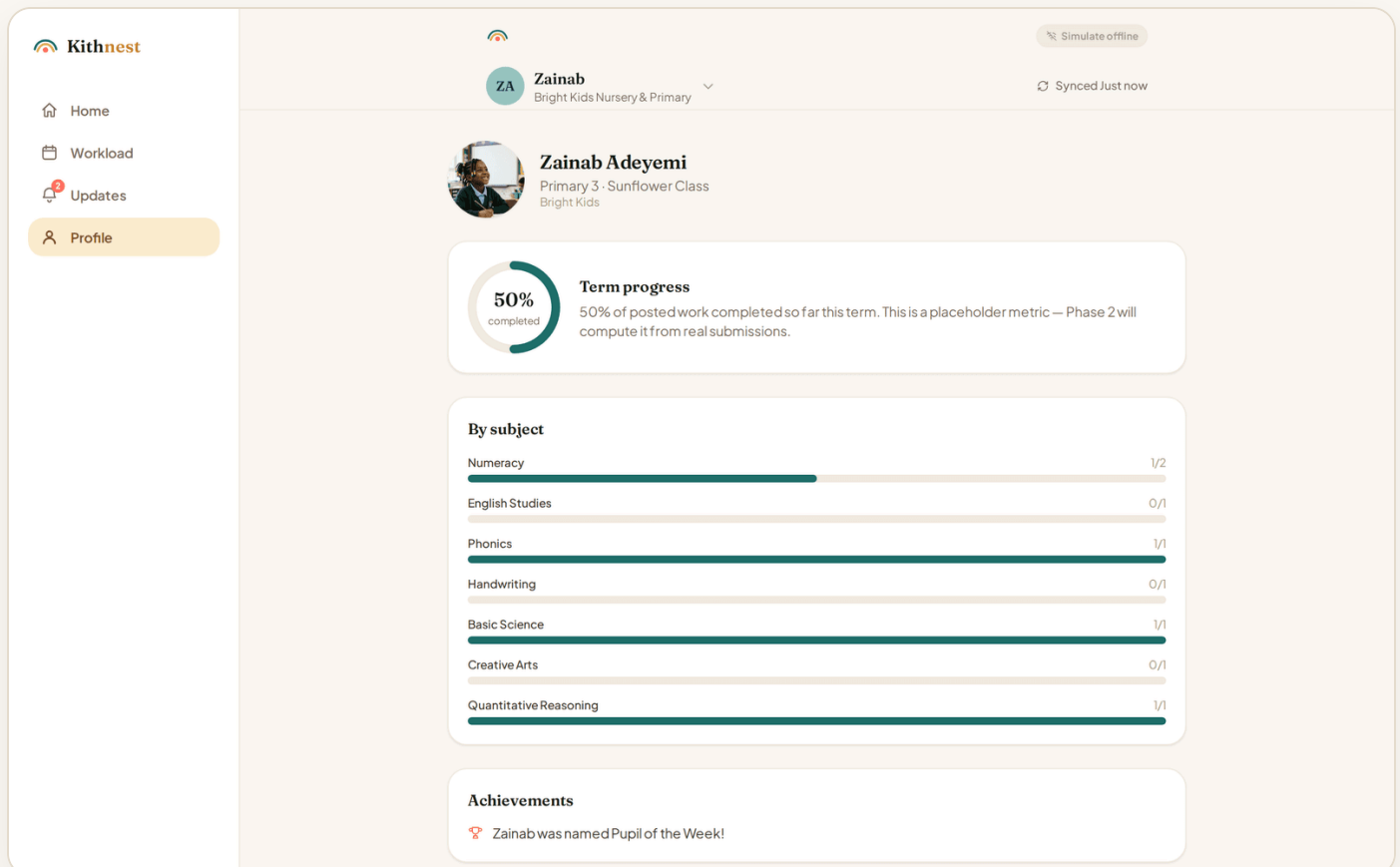


WALKTHROUGH · PART 3

Updates that reach parents, and progress that builds



Updates that meet parents where they are. School announcements show up in a simple, chronological feed. The ones marked **SMS** are labelled that way deliberately, because that's how they'd actually reach a parent who isn't checking the app that day. Unread updates stay visibly marked until opened.



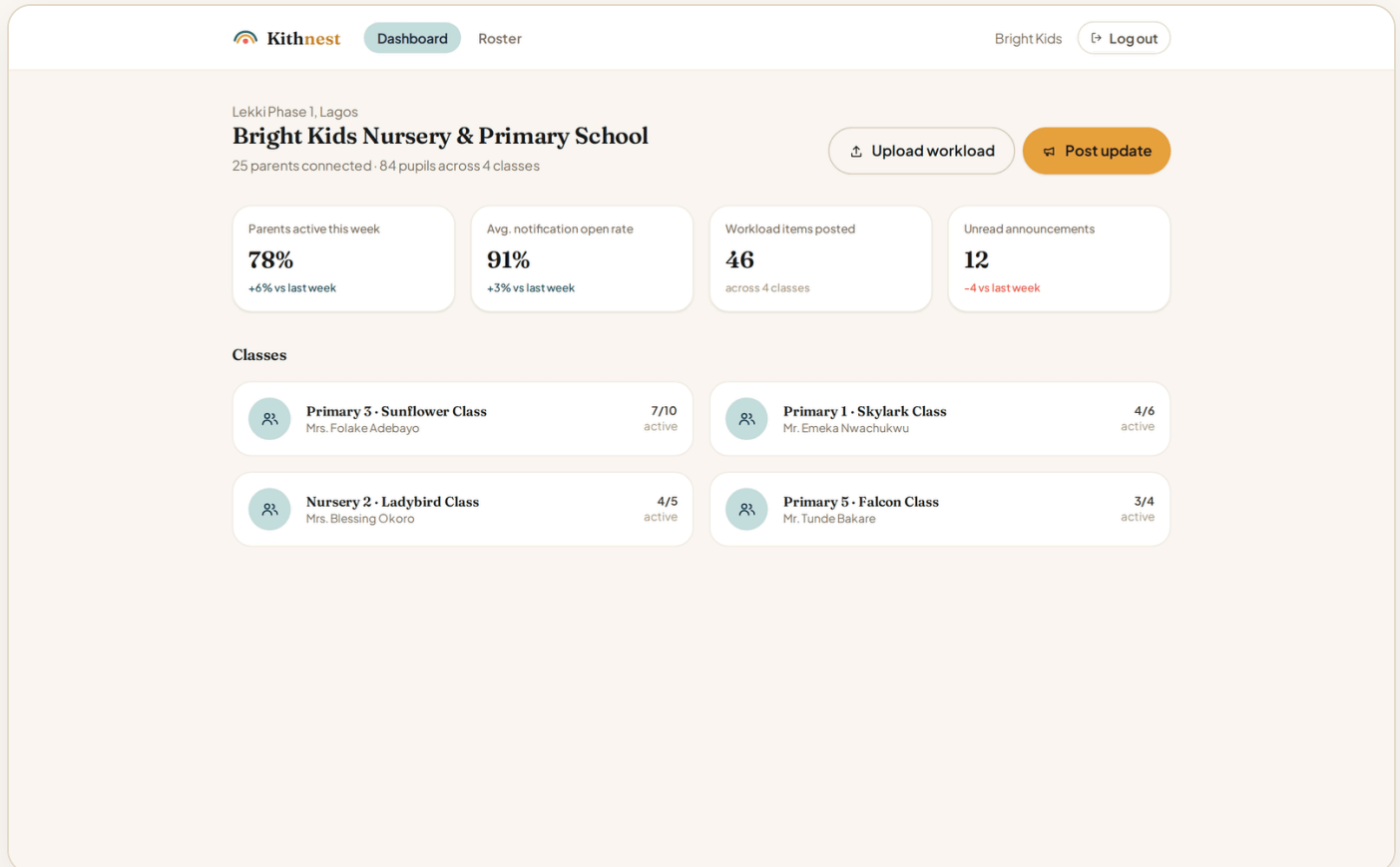
Watching progress build. A simple profile view gives a parent a sense of how their child is doing across subjects, a term-progress ring and a per-subject breakdown, without turning into a spreadsheet only a teacher could read.



WALKTHROUGH · THE OTHER SIDE

What the school sees

Built with deliberately less design weight than the parent side, because in this product the parent is the star, not the school office. Still, admins need a clear, fast way to see engagement and post updates.



A dashboard, not a chore. Admins see how many parents are actively engaging, how many workload items have gone out, and can post a new update or upload new workload in a couple of taps.

Parent roster

Connected parents across every class.

All classes

Sunflower Class

Skylark Class

Ladybird Class

Falcon Class

Search parent or pupil...

Mrs. Amaka Adeyemi Parent of Zainab Adeyemi	Sunflower Class	0803 214 7765	Today	Active
Mr. & Mrs. Nwosu Parent of Chiamaka Nwosu	Sunflower Class	0705 662 1190	Today	Active
Alhaji Suleiman Bello Parent of Ibrahim Suleiman	Sunflower Class	0812 340 5521	Yesterday	Active
Mrs. Temitope Ogunleye Parent of Damilola Ogunleye	Sunflower Class	0908 774 3302	Yesterday	Active
Mr. Kelechi Okonkwo Parent of Kelechi Okonkwo Jr.	Sunflower Class	0803 991 4487	3 weeks ago	Inactive
Mrs. Fatima Abubakar Parent of Amina Abubakar	Sunflower Class	0706 128 8890	2 days ago	Active
Mr. Chukwuemeka Eze Parent of Chidinma Eze	Sunflower Class	0813 220 7761	Never logged in	Invited
Mrs. Grace Effiong Parent of Emmanuel Effiong	Sunflower Class	0902 341 6675	3 days ago	Active
Mr. Segun Afolabi Parent of Tobì Afolabi	Sunflower Class	0705 118 2290	Today	Active
Mrs. Halima Mohammed	Sunflower Class	0816 552 0043	Never logged in	Invited

Knowing who's connected. A simple roster shows which parents are active, which haven't joined yet, and which have gone quiet, organised by class, with search built in.

WHY IT LOOKS THIS WAY

Why it looks the way it does

We deliberately avoided the generic "SaaS dashboard" look, white cards, corporate blue, stock icons, because that reads as an enterprise tool, not something built for a family. Every visual choice ties back to the product's identity: warmth, trust, and the connection between home and school.

- **A warm colour palette**, marigold and deep teal rather than corporate blue, that feels closer to a consumer app parents already trust than an enterprise tool.
- **A distinct typeface pairing** with real character, not the default look nearly every startup reaches for.
- **A hand-built icon set**, so nothing feels borrowed from a generic template.
- **Small, purposeful motion**, things gently sliding, fading, or settling into place, that makes the app feel alive without ever slowing anyone down.
- **Real, candid photography** rather than generic stock photos or AI-generated faces, sourced specifically for this product.

The goal was simple: this should feel like a product a real design team spent real time on, because that's exactly what happened. For a product where trust and warmth matter as much as function, how it feels is part of what we're building, not a finishing touch.

HOW IT'S BUILT, PHASE 1 THROUGH PHASE 3

The stack, at a glance

A quick reference before the detail on the following pages

LAYER	PHASE 1 (TODAY)	PHASE 2 (NEXT)	PHASE 3 (LATER)
Frontend	React + TypeScript, built and working	Same frontend, connected to real data	Same frontend, plus billing & analytics views
Backend	None — mock data in the browser	FastAPI (Python) application layer	Same, extended with AI & billing services
Database & auth	Browser storage only	Supabase — PostgreSQL + phone-OTP login	Same, with richer usage history feeding AI
Messaging (SMS)	Simulated — labels only	Twilio — real SMS + OTP login	Expanded SMS packages for larger schools
Hosting	Local development only	Render (or an equivalent platform)	Same, scaled as more schools onboard
AI	Not present	Not present	Hosted LLM (e.g. Claude) — parent & school insights
Payments	Not present	Not present	Paystack / Flutterwave — subscriptions & fees

Nothing below is locked in stone this early, but it reflects real decisions, not placeholders, so engineering can plan against it and the product team knows what each phase actually depends on.

PHASE 1 · RUNNING TODAY

The frontend, built and working

What's actually powering the product you can click through

Phase 1 is a complete, modern web frontend, the same category of technology real consumer apps are built with, just not yet connected to a live backend.

- **React + TypeScript.** The interface is built as reusable components, and TypeScript catches a large class of bugs before they ever reach a phone screen, by checking that data is shaped the way the code expects, before it ships.
- **Tailwind CSS, with a custom design system.** Rather than pulling in a generic component library, the colours, spacing, and type scale are defined once and reused everywhere, so the product looks intentional rather than assembled from templates.
- **Framer Motion.** The small animations, cards settling into place, a progress ring filling in, are handled by a purpose-built animation library, kept deliberately subtle so nothing ever slows a user down on an older phone.
- **Mobile-first and bandwidth-conscious from day one.** Fonts are trimmed to only the weights actually used, images are compressed and served in modern formats, and heavy interactions are avoided. The product has to work well on a mid-range Android phone on 3G, not just a demo laptop on office Wi-Fi.

Where the data comes from right now: every school, pupil, parent, and message you see is realistic sample data stored in the browser itself. There is no server, and nothing typed into the app is sent anywhere. That's exactly what Phase 2 replaces.

PHASE 2 · PLANNED NEXT

Giving the app something real to talk to

The backend and data layer that replace the mock data

Phase 2's job is to replace the sample data with an actual backend: a system that stores real schools, pupils, and parents, and enforces the rules about who can see what.

- **FastAPI (Python) as the application layer.** This is where Kithnest-specific rules live, a parent can only ever see the pupils linked to their account, a school can only ever see its own classes and parents, an admin action is validated and logged before anything is posted.
- **Supabase as the managed data platform.** Supabase provides a production-grade PostgreSQL database, user authentication, and file storage without needing to run and secure that infrastructure ourselves, letting a small team move fast without cutting corners on reliability.
- **Multi-tenancy from the ground up.** Every school's data is isolated from every other school's at the database level, not just hidden in the interface, using PostgreSQL's row-level security. This was designed into the data model from Phase 1 specifically so it doesn't need retrofitting later; adding tenant isolation after the fact is one of the most expensive mistakes a growing product can make.
- **Real offline support.** The Phase 1 offline banner was a preview. In Phase 2, the app actually caches a parent's data on their device and reconciles it with the server the moment connectivity returns, so a parent never sees a blank screen because of a dropped connection.



PHASE 2 · PLANNED NEXT

Reaching parents for real, and going live

Messaging, hosting, and the path to production

- **Twilio for real SMS delivery.** This replaces the simulated "SMS" labels in Phase 1 with actual text messages for urgent updates. It also doubles as the login mechanism: a parent verifies their phone number with a one-time code sent by SMS, so the "phone number, no password" login already built in Phase 1 becomes fully real with no redesign needed.
- **Kept behind an internal abstraction.** The messaging provider is wrapped so it can be swapped or supplemented later, a bigger school might warrant a different provider or pricing tier, without touching the rest of the product.
- **Hosting on Render (or an equivalent platform, such as Railway or Fly.io).** These platforms let us deploy, update, and scale the app without a dedicated infrastructure team, appropriate for where the product is right now, with room to move to something more heavyweight later, once scale genuinely demands it.
- **Separate staging and production environments.** Once real schools are involved, changes are tested in a private environment before they ever reach a live school, so day-to-day development never risks real parent or pupil data.

Choosing Twilio for SMS lines up neatly with choosing Supabase for authentication, Supabase's phone-based login is built to work with providers like Twilio directly, so the login pattern already designed in Phase 1 and the messaging channel arrive together, not as two separate integrations.



PHASE 3 · LATER

Making it smarter, and sustainable

Intelligence, monetisation, and scale

- **AI-powered insights, for both audiences.** Parents get plain-language summaries of how their child is trending over time (for example, noticeably stronger in Numeracy this term, worth a bit more practice in Phonics). Schools get early signals when a pupil, class, or parent seems to be falling behind or disengaging. Both are generated from the real usage data collected since Phase 2, not a separate system layered on top.
- **Built on a hosted large language model, such as Claude.** Rather than training our own model, Kithnest calls a proven, existing AI system with the school's own data, scoped by the exact same per-school privacy boundaries as everything else in the product.
- **Data protection is a hard requirement here, not a nice-to-have.** This is children's data, in a Nigerian context, so staying mindful of data-protection rules such as the NDPR (Nigeria Data Protection Regulation) shapes how AI features are built from the outset, not bolted on afterward.
- **Billing, through a Nigerian-established provider** (Paystack or Flutterwave). This is what turns school subscriptions, and later, digital fee payments, into a real, working part of the business, kept behind its own service boundary so billing logic never leaks into or complicates the core product.
- **Custom branding and expanded SMS packages.** Larger schools or school groups can get their own visual identity on the parent-facing app, and bigger messaging volumes, without any change to the underlying architecture.

READ THIS BEFORE DEMOING IT YOURSELF

What's real, and what's simulated, right now

Everything you click responds the way it would in a real product. This table is the honest map of what's actually happening behind each interaction today, in Phase 1, specifically.

FEATURE	WHAT YOU'LL SEE	WHAT'S ACTUALLY HAPPENING
Login (parent & school)	Feels real, validates a school code	No real accounts yet; any matching demo code works
Dashboard & workload data	Realistic Nigerian names, subjects, dates	Pre-loaded sample data, not a live database
Notifications	Full read/unread behaviour, persists between visits	Same sample notifications every visit; nothing new arrives
SMS labels	Clearly marked which updates would go by text	No text messages are actually sent (that's Twilio, Phase 2)
Offline banner	Fully working toggle and visual state	Simulated; not yet tied to real connectivity
School admin actions	Full flow, including a "sent" confirmation	Nothing is actually delivered anywhere yet

Nothing here is a shortcut we're hiding. It's the deliberate scope of Phase 1: prove the experience before building the infrastructure described on the previous pages.



LOOKING AHEAD

What comes next

Phase 2 is about becoming real: real school and parent accounts, a live database, genuine offline syncing, and actual SMS delivery, the backend and data foundations described on the previous pages, built out.

Phase 3 is about becoming a business: AI-driven insight for parents and schools, subscription billing, custom branding for larger school groups, and expanded messaging for scale.

Phase 1's job was narrower, and we think more important right now: prove the experience is right before spending engineering time making it live.

How to review it yourself

The build is fully clickable and lives in our GitHub repository. Both the parent and school sign-in screens come pre-filled with working demo credentials, so you can walk through either journey end to end in a couple of minutes without needing anything set up for you.

Try both journeys: **parent** (sign in → dashboard → workload → updates → profile) and **school admin** (sign in → dashboard → roster). Everything should feel complete; nothing should dead-end.